Introduction to R | *Introduction à R*

Vincent, Yannick

2026-06-10

Section 1

Guide

R refresher

Rappel R

- Objects
- Functions

- Objets
- Fonctions

Section 2

Getting started

Overview
Vue d'ensemble

This section is for readers opening R for the first time. It covers the practical setup the rest of the course assumes.

Cette section s'adresse à ceux qui ouvrent R pour la première fois. Elle couvre la configuration pratique que le reste du cours présuppose.

R and RStudio
R et RStudio

R is the programming language and computational engine.
RStudio is a separate application that provides a more comfortable interface to R.
Install R first, then RStudio.

R est le langage et le moteur de calcul.
RStudio est une application distincte qui offre une interface plus confortable à R.
Installez R d'abord, puis RStudio.

RStudio panes

Les fenêtres de RStudio

When you open RStudio you typically see four panes:

- **Console** — run commands one at a time; results print here
- **Source / Editor** — write scripts (`.R`) or Quarto (`.qmd`); code runs only when you send it

À l'ouverture de RStudio, vous voyez typiquement quatre fenêtres :

- **Console** — exécuter des commandes une par une ; les résultats s'affichent ici
- **Source / Éditeur** — écrire des scripts (`.R`) ou Quarto (`.qmd`) ; le code ne s'exécute que lorsque vous l'envoyez

RStudio panes (2)

Les fenêtres de RStudio (2)

- **Environment / History** — objects in memory and command history
- **Files / Plots / Help / Packages** — file browser, plots, help pages, installed packages

- **Environnement / Historique** — objets en mémoire et historique des commandes
- **Fichiers / Graphiques / Aide / Packages** — explorateur de fichiers, graphiques, aide, packages installés

Running code
Exécuter du code

Type in the **console** and press Enter, or write in a **script** and send with **Ctrl + Enter** (Windows/Linux) or **Cmd + Enter** (Mac).
Scripts are preferable — work is saved and reproducible.

Tapez dans la **console** et appuyez sur Entrée, ou écrivez dans un **script** et envoyez avec **Ctrl + Entrée** (Windows/Linux) ou **Cmd + Entrée** (Mac).
Les scripts sont préférables — le travail est sauvegardé et reproductible.

Assignment vs display

Affectation vs affichage

Two kinds of commands behave differently:

Deux types de commandes se comportent différemment :

```
2 + 2                # add two numbers and show the result
```

```
[1] 4
```

```
x <- 2 + 2           # compute 2 + 2 and store in object x
x                    # show the value stored in x
```

```
[1] 4
```

Working directory
Répertoire de travail

The **working directory** is the folder R reads from and writes to by default. Many early errors come from R looking in the wrong place for a data file.

Le **répertoire de travail** est le dossier que R utilise par défaut pour lire et écrire. Beaucoup d'erreurs viennent d'un mauvais chemin vers un fichier de données.

```
getwd()              # current working directory
```

```
[1] "C:/WZB Dropbox/Macartan Humphreys/LDID 2026/Slides_abidjan_2026"
```

```
list.files("data/")  # files R can see in a folder
```

```
[1] "echocardiogram.data" "ships.csv"           "ships_out.csv"
```

RStudio projects
Projets RStudio

Best practice: work inside an **RStudio Project**.

1. File → New Project → New Directory → New Project
2. Put scripts and data inside the project folder
3. Open the project each session — `getwd()` should point to the project root

Bonne pratique : travailler dans un **projet RStudio**.

1. Fichier → Nouveau projet → Nouveau répertoire → Nouveau projet
2. Placer scripts et données dans le dossier du projet
3. Ouvrir le projet à chaque session — `getwd()` doit pointer vers la racine du projet

R's base installation is extended by **packages**. **Install** once; **load** each session with `library()`.

L'installation de base de R est étendue par des **packages**. **Installer** une fois ; **charger** à chaque session avec `library()`.

```
install.packages("readr")   # download and install (once)
library(readr)              # load for this session
```

Getting help
Obtenir de l'aide

Read error messages from the **bottom up** — they usually identify the object or file that caused the problem.

Lisez les messages d'erreur **de bas en haut** — ils identifient en général l'objet ou le fichier en cause.

```
?mean                    # help page for mean()
args(mean)               # arguments that mean() accepts
```

Housekeeping

Nettoyage de la session

```
ls()                            # list objects in memory
```

```
[1] "x"
```

```
rm(x)                           # remove object x
rm(list = ls())                 # remove all objects (use with care!)
```

Section 3

Scalars, vectors, and arithmetic

R as a calculator

R comme calculatrice

R works as a calculator. Assignment uses <- (or =). Common math functions are built in.

R fonctionne comme une calculatrice. L'affectation utilise <- (ou =). Les fonctions mathématiques courantes sont intégrées.

```
r <- 3
pi * r^2                    # area of a circle
```

```
[1] 28.27433
```

```
sin(pi / 4)
```

```
[1] 0.7071068
```

```
sqrt(2)
```

```
[1] 1.414214
```

```
log(2)
```

```
[1] 0.6931472
```

Creating vectors

Créer des vecteurs

Vectors are created with `c()`. Arithmetic is **element-wise**.

Les vecteurs se créent avec `c()`. L'arithmétique est **élément par élément**.

```
x <- c(2, 4, 6, -1)
y <- c(4, 1, 2, 2)

x + y
```

```
[1] 6 5 8 1
```

```
x - y
```

```
[1] -2  3  4 -3
```

```
x / y
```

```
[1]  0.5  4.0  3.0 -0.5
```

```
x * y
```

```
[1]  8  4 12 -2
```

Vector summaries

Résumés d'un vecteur

```
length(x)
```

```
[1] 4
```

```
mean(x)
```

```
[1] 2.75
```

```
sd(x)
```

```
[1] 2.986079
```

```
var(x)
```

```
[1] 8.916667
```

```
z <- c("A", "B", "a", "c")
z[1]
```

```
[1] "A"
```

```
z[2:3]
```

Section 4

Data frames

What is a data frame?

Qu'est-ce qu'un data frame ?

Most real data lives in **data frames**: tables of rows (cases) and columns (variables). Columns can differ in type.

La plupart des données sont des **data frames** : tableaux de lignes (cas) et colonnes (variables). Les colonnes peuvent être de types différents.

```
height <- c(172, 165, 180, 158)
weight <- c(70, 60, 82, 55)
sex    <- c("F", "F", "M", "F")

people <- data.frame(height, weight, sex)
people
```

```
  height weight sex
1    172     70   F
2    165     60   F
3    180     82   M
4    158     55   F
```

Accessing columns

Accéder aux colonnes

```
people$height
```

```
[1] 172 165 180 158
```

```
people[ , "height"]
```

```
[1] 172 165 180 158
```

```
people[ , 1]
```

```
[1] 172 165 180 158
```

```
people[1:2, ]
```

```
  height weight sex
1    172     70   F
2    165     60   F
```

```
people[1:2, 1]
```

```
[1] 172 165
```

Section 5

Reading data

Reading a CSV
Lire un CSV

Put the data file in your project folder, then use `read.csv()` or `readr::read_csv()`.

Placez le fichier dans le dossier du projet, puis utilisez `read.csv()` ou `readr::read_csv()`.

```
ships <- read.csv("data/ships.csv")

class(ships)
```

```
[1] "data.frame"
```

```
names(ships)
```

```
[1] "Type"         "Construction" "Operation"    "Months"       "Dammage"
```

```
head(ships)
```

```
  Type Construction Operation Months Dammage
1    A           60         60    127       0
2    A           60         75     63       0
3    A           65         60   1095       3
4    A           65         75   1095       4
5    A           70         60   1512       6
```

Missing values on import

Valeurs manquantes à l'import

```
Echo <- read.csv("data/echocardiogram.data",
                 header      = FALSE,
                 na.strings = c("?"))

names(Echo) <- c("survival", "alive", "age", "peri",
                 "frac_short", "epss", "lvdd")

head(Echo)
```

```
  survival alive age peri frac_short   epss  lvdd <NA> <NA>  <NA> <NA> <NA>
1       11     0  71    0      0.260  9.000 4.600   14 1.00 1.000 name    1
2       19     0  72    0      0.380  6.000 4.100   14 1.70 0.588 name    1
3       16     0  55    0      0.260  4.000 3.420   14 1.00 1.000 name    1
4       57     0  60    0      0.253 12.062 4.603   16 1.45 0.788 name    1
5       19     1  57    0      0.160 22.000 5.750   18 2.25 0.571 name    1
6       26     0  68    0      0.260  5.000 4.310   12 1.00 0.857 name    1
  <NA>
1    0
```

Writing data

Exporter des données

```
write.csv(ships,
          file = "data/ships_out.csv",
          row.names = FALSE)
```

Section 6

Data types

Factors

Facteurs (variables catégorielles)

Categorical variables are stored as **factors**. Inspect levels with `levels()` and counts with `table()`.

Les variables catégorielles sont des **facteurs**. Inspectez les niveaux avec `levels()` et les effectifs avec `table()`.

```
ships$Type      <- as.factor(ships$Type)
ships$Operation <- as.factor(ships$Operation)

levels(ships$Operation)
```

```
[1] "60" "75"
```

```
table(ships$Operation)
```

```
60 75
20 20
```

Re-leveling factors

Recoder les niveaux

Use `relevel()` to change the **reference category**.

Utilisez `relevel()` pour changer la **catégorie de référence**.

```
ships$Operation <- relevel(ships$Operation, ref = "75")
levels(ships$Operation)
```

```
[1] "75" "60"
```

Factor codes vs labels

Codes vs étiquettes

Warning: `as.numeric()` on a factor returns underlying **codes**, not the labels.

Attention : `as.numeric()` sur un facteur renvoie les **codes** internes, pas les étiquettes.

```
codes  <- as.numeric(ships$Operation)
labels <- as.character(ships$Operation)

codes[1:5]
```

```
[1] 2 1 2 1 2
```

```
labels[1:5]
```

```
[1] "60" "75" "60" "75" "60"
```

Section 7

Summary statistics

`summary()` gives an overview of each variable in a data frame.

`summary()` donne un aperçu de chaque variable d'un data frame.

```
summary(ships)
```

```
 Type   Construction   Operation     Months          Dammage
 A:8   Min.   :60.00   75:20     Min.   :    0.0   Min.   : 0.0
 B:8   1st Qu.:63.75   60:20     1st Qu.:  175.8   1st Qu.: 0.0
 C:8   Median :67.50             Median :  782.0   Median : 2.0
 D:8   Mean   :67.50             Mean   : 4089.3   Mean   : 8.9
 E:8   3rd Qu.:71.25             3rd Qu.: 2078.5   3rd Qu.:11.0
       Max.   :75.00             Max.   :44882.0   Max.   :58.0
```

Numeric summaries

Résumés numériques

```
mean.dam <- mean(ships$Dammage)
sd.dam   <- sd(ships$Dammage)
var.dam  <- var(ships$Dammage)

quantile(ships$Dammage,
         probs = c(0.25, 0.5, 0.75))
```

```
25% 50% 75%
  0   2  11
```

Covariance and correlation

Covariance et corrélation

```
cov(ships$Months, ships$Dammage)
```

```
[1] 115306.1
```

```
cor(ships$Months, ships$Dammage)
```

```
[1] 0.8525174
```

Tables and missing values

Tableaux et valeurs manquantes

```
table(ships$Type)
```

```
A B C D E
8 8 8 8 8
```

```
table(ships$Operation, ships$Type)
```

```
    A B C D E
  75 4 4 4 4 4
  60 4 4 4 4 4
```

```
mean(Echo$age)
```

```
[1] NA
```

```
mean(Echo$age, na.rm = TRUE)
```

```
[1] 62.81372
```

Section 8

Plotting with ggplot2

The ggplot pattern

Le modèle ggplot

We use **ggplot2** (tidyverse). Basic pattern:

Nous utilisons **ggplot2** (tidyverse). Modèle de base :

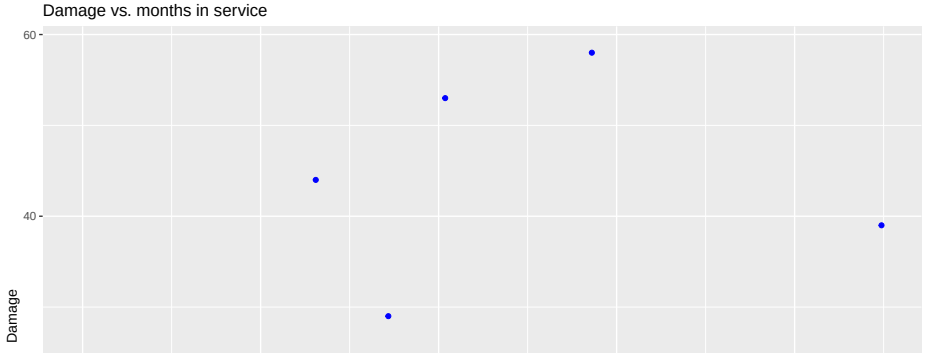
```
ggplot(data = ..., aes(x = ..., y = ...)) +
  geom_...()
```

```
library(ggplot2)
```

Scatterplot

Nuage de points

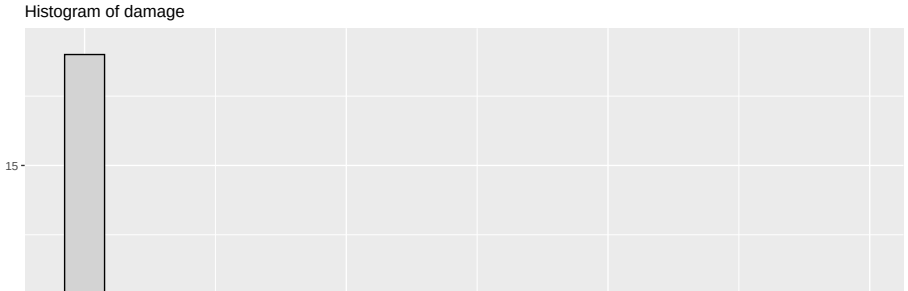
```
ggplot(data = ships,
       aes(x = Months, y = Dammage)) +
  geom_point(color = "blue") +
  labs(x = "Months",
       y = "Damage",
       title = "Damage vs. months in service")
```



Histogram

Histogramme

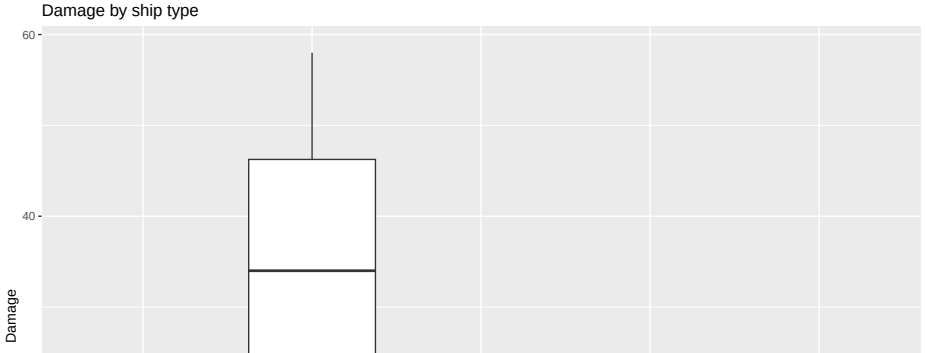
```
ggplot(data = ships,
       aes(x = Dammage)) +
  geom_histogram(bins = 20,
                 fill = "lightgray",
                 color = "black") +
  labs(x = "Damage",
       y = "Count",
       title = "Histogram of damage")
```



Boxplot by group

Boîte à moustaches par groupe

```
ggplot(data = ships,
       aes(x = Type, y = Dammage)) +
  geom_boxplot() +
  labs(x = "Ship type",
       y = "Damage",
       title = "Damage by ship type")
```



Mini workflow

Mini parcours

1. Open the RStudio project
2. Create a script `intro.R`
3. Read the data
4. Inspect variables
5. Summarize and plot with ggplot

1. Ouvrir le projet RStudio
2. Créer un script `intro.R`
3. Importer les données
4. Inspecter les variables
5. Résumer et graphiquer avec ggplot

Mini workflow (code)

Mini parcours (code)

```
ships <- read.csv("data/ships.csv")

str(ships)
```

```
'data.frame':   40 obs. of  5 variables:
 $ Type        : chr  "A" "A" "A" "A" ...
 $ Construction: int  60 60 65 65 70 70 75 75 60 60 ...
 $ Operation   : int  60 75 60 75 60 75 60 75 60 75 ...
 $ Months      : int  127 63 1095 1095 1512 3353 0 2244 44882 17176 ...
 $ Dammage     : int  0 0 3 4 6 18 0 11 39 29 ...
```

```
summary(ships$Dammage)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
    0.0     0.0     2.0     8.9    11.0    58.0
```

```
ggplot(data = ships,
       aes(x = Months, y = Dammage)) +
  geom_point(color = "darkred") +
```

Section 9

Let's go

Simulation by hand

Simulation à la main

A tiny randomized experiment in base R:

Une petite expérience randomisée en R de base :

```
N <- 10
Z <- sample(0:1, N, replace = TRUE)
Y <- Z + rnorm(N)

lm(Y ~ Z) |> summary()
```

```
Call:
lm(formula = Y ~ Z)

Residuals:
    Min      1Q  Median      3Q     Max
-1.2992 -0.5770  0.1619  0.3835  1.7235

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept)  -0.2003     0.4833  -0.414   0.6895
```

Same in DeclareDesign

Même chose avec DeclareDesign

The same design declared with **DeclareDesign**:

La même conception déclarée avec **DeclareDesign** :

```
library(DeclareDesign)
```

```
N <- 10
design <-
  declare_model(N,
                Z = sample(0:1, N, replace = TRUE),
                Y = Z + rnorm(N)) +
  declare_inquiry(ATE = 1) +
  declare_estimator(Y ~ Z)

design
```

```
Research design declaration summary

Step 1 (model): declare_model(N, Z = sample(0:1, N, replace = TRUE), Y = Z + rnorm
```